

Lập trình Java

Bean trong spring mvc

GV Nguyễn Đức Kiên



Nội dung bài học

- I. IOC (Inversion of Control)
- II. DI (Dependency Injection)
- III. Beans
- IV. @Autowired
- V. Các Loại Annotation trong Spring
- VI. JDBC Template

I) IOC (Inversion of Control)

- Trong Spring, Spring Container (IoC Container) sẽ tạo các đối tượng, lắp ráp chúng lại với nhau, cấu hình các đối tượng và quản lý vòng đời của chúng từ lúc tạo ra cho đến lúc bị hủy.
- Spring container sử dụng DI để quản lý các thành phần, đối tượng để tạo nên 1 ứng dụng. Các thành phần, đối tượng này gọi là Spring Bean(các object java)
- Để tạo đối tượng, cấu hình, lắp ráp chúng, Spring Container sẽ đọc thông tin từ các file xml và thực thi chúng.



- IoC Container trong Spring có 2 kiểu là **BeanFactory** và **ApplicationContext**

1.2) BeanFactory

- BeanFactory là giao diện cơ bản nhất trong Spring, cung cấp chức năng để lấy các bean từ container.
- Đây là cách tiếp cận nhẹ nhàng để quản lý bean, nhưng không cung cấp đầy đủ các tính năng mạnh mẽ như ApplicationContext
- Chức năng: BeanFactory chỉ cung cấp các phương thức cơ bản để truy xuất bean từ container thông qua tên hoặc kiểu. Nó chủ yếu được sử dụng trong các trường hợp yêu cầu ít tính năng và tối ưu tài nguyên.
- Sử dụng: Nó thường được sử dụng trong những ứng dụng có yêu cầu nhẹ và không cần tính năng nâng cao như sự kiện, hỗ trợ AOP, hoặc quốc tế hóa. Ví dụ: XmlBeanFactory (cũ, không còn sử dụng trong Spring 3.x trở đi) hoặc DefaultListableBeanFactory.

1.2) BeanFactory

- Code ví dụ:

```
public class HelloWorld {  
    private String message;  
  
    public void setMessage(String message) {  
        this.message = message;  
    }  
  
    public void getMessage() {  
        System.out.println("Print : " + message);  
    }  
}
```

```
<?xml version = "1.0" encoding = "UTF-8"?>  
  
<beans xmlns = "http://www.springframework.org/schema/beans"  
    xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation = "http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans-3.0.xsd">  
  
    <bean id = "helloWorld" class = "stackjava.com.springioc.beanfactory.HelloWorld"  
        <property name = "message" value = "Hello World!"/>  
    </bean>  
  
</beans>
```

```
// tạo factory  
DefaultListableBeanFactory factory = new DefaultListableBeanFactory();  
  
// đọc thông tin file cấu hình và gán vào factory  
XmlBeanDefinitionReader reader = new XmlBeanDefinitionReader(factory);  
reader.loadBeanDefinitions(new ClassPathResource("beans.xml"));  
  
//tạo đối tượng từ factory  
HelloWorld obj = (HelloWorld) factory.getBean("helloWorld");  
obj.getMessage();
```

1.3) ApplicationContext

- ApplicationContext là một giao diện mở rộng của BeanFactory và cung cấp tất cả các tính năng của BeanFactory nhưng bổ sung thêm nhiều tính năng mạnh mẽ hơn, như sự kiện, hỗ trợ AOP, quốc tế hóa và nhiều tính năng khác.
- Chức năng: ApplicationContext không chỉ cho phép truy xuất bean mà còn cung cấp các khả năng mở rộng như:
 - Hỗ trợ cho các sự kiện (Event Handling).
 - Cung cấp các phương thức hỗ trợ AOP.
 - Quốc tế hóa (Internationalization).
 - Tự động phát hiện các bean trong các cấu hình như @ComponentScan.

1.3) ApplicationContext

- Sử dụng: ApplicationContext thường được sử dụng trong hầu hết các ứng dụng Spring vì tính năng phong phú và dễ dàng mở rộng.
- Các lớp triển khai của ApplicationContext bao gồm:
 - **ClassPathXmlApplicationContext:** Đọc cấu hình XML từ classpath.
 - **AnnotationConfigApplicationContext:** Được sử dụng khi cấu hình ứng dụng với annotation thay vì XML.
 - **GenericWebApplicationContext:** Được sử dụng cho các ứng dụng web.

1.3) ApplicationContext

- Code ví dụ:

1) Ví dụ class DataSource.java chứa thông tin kết nối tới database.

```
public class DataSource {  
    private String driverClassName;  
    private String url;  
    private String username;  
    private String password;
```

2) Tạo file
applicationContext.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:p="http://www.springf  
    xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.spring  
  
    <bean id="dataSource" class="stackjava.com.springioc.applicationcontext.DataRes  
        <property name="driverClassName" value="com.mysql.jdbc.Driver" />  
        <property name="url" value="jdbc:mysql://localhost/database_name" />  
        <property name="username" value="root" />  
        <property name="password" value="admin1234" />  
    </bean>  
  
</beans>
```


1.3) ApplicationContext

- Tạo hàm main

```
public static void main(String[] args) {  
    new *  
    ApplicationContext context = new ClassPathXmlApplicationContext( configLocation: "applicationContext.xml");  
    DataResource obj = (DataResource) context.getBean( name: "dataResource");  
    obj.printConnection();  
}
```

- Kết quả:

```
url: jdbc:mysql://localhost/database_name  
username/password: root/admin1234
```

- Bây giờ muốn thay đổi message trong đối tượng HelloWorld, hay database chỉ cần thay đổi username/password hay đổi kết nối sang database khác chỉ cần đổi lại thông tin trong file config.xml là đã thay đổi được luồng chạy của chương trình, đó chính là 1 trong số nhiều ứng dụng của IoC.

II) DI (Dependency Injection)

- Dependency Injection (DI) là một mẫu thiết kế trong lập trình phần mềm, giúp tách biệt các phụ thuộc giữa các đối tượng và các thành phần của hệ thống.
- Mục tiêu của DI là giảm sự phụ thuộc giữa các đối tượng, khiến mã nguồn dễ bảo trì, dễ kiểm thử và dễ mở rộng hơn.
- Trong DI, **các phụ thuộc (dependencies) của một đối tượng** được tiêm vào (injected) thay vì tự khởi tạo chúng.
- Điều này giúp tách biệt việc khởi tạo các đối tượng với việc sử dụng chúng, và thay vào đó, chúng sẽ được cung cấp (tiêm) từ bên ngoài thông qua một cơ chế quản lý (thường là container trong các framework như Spring).

Các hình thức của Dependency Injection:

- **Constructor Injection:** Phụ thuộc được truyền vào qua constructor của lớp

```
public class UserServiceImpl implements UserService { 2 usages

    private UserDao userDao; 2 usages
    private RoleDao roleDao; 2 usages

    public UserServiceImpl(UserDao userDao, RoleDao roleDao) { 1
        this.userDao = userDao;
        this.roleDao = roleDao;
    }
```


Các hình thức của Dependency Injection:

- **Setter Injection:** Phụ thuộc được truyền vào thông qua setter methods.

```
public class UserServiceImpl implements UserService {  
  
    private UserDao userDao; 3 usages  
    private RoleDao roleDao; 3 usages  
  
    public void setRoleDao(RoleDao roleDao) { no usages ne  
        this.roleDao = roleDao;  
    }  
  
    public void setUserDao(UserDao userDao) { no usages ne  
        this.userDao = userDao;  
    }  
}
```

Mối liên hệ giữa DI và IOC:

- IOC là một nguyên lý rộng, trong đó Dependency Injection (DI) là một kỹ thuật cụ thể giúp thực hiện IOC.
- DI là một phương thức cụ thể để lật ngược (invert) sự kiểm soát của các phụ thuộc trong hệ thống.
- Thay vì các đối tượng tự tạo và quản lý các phụ thuộc của chúng, IOC container sẽ quản lý và "tiêm" các phụ thuộc vào đối tượng khi cần thiết.
- **Tóm lại:**
 - ❑ IOC (Inversion of Control) là một nguyên lý thiết kế, nơi việc kiểm soát việc tạo và quản lý đối tượng được chuyển giao cho một container hoặc framework.
 - ❑ DI (Dependency Injection) là một kỹ thuật trong IOC, trong đó các đối tượng phụ thuộc được tiêm vào đối tượng cần sử dụng thay vì đối tượng đó tự tạo các phụ thuộc của mình.
 - ❑ Ví dụ trong Spring Framework, IOC container sẽ quản lý các bean và tiêm các phụ thuộc vào các bean thông qua DI (constructor, setter hoặc field injection).

III) Beans

- Trong Spring MVC, bean là một đối tượng mà Spring quản lý trong container của mình. Mỗi bean là một instance(object) của một class được định nghĩa trong file cấu hình Spring hoặc qua annotations, và container Spring sẽ đảm nhận việc quản lý vòng đời của bean đó.
- **Khái niệm về Bean trong Spring:**
 - ☐ Bean là một đối tượng được quản lý bởi container Spring.
 - ☐ Mỗi bean có một scope (phạm vi), nghĩa là mỗi bean sẽ có một vòng đời nhất định.
 - ☐ Bean có thể được tạo một lần và tái sử dụng, hoặc có thể được tạo mới mỗi khi yêu cầu.
 - ☐ Spring container sẽ tự động khởi tạo và quản lý các bean trong ứng dụng của bạn, từ việc tiêm phụ thuộc đến việc xử lý vòng đời của chúng.

3.1) Các loại Bean Scopes trong Spring

- Spring cung cấp nhiều loại bean scopes để bạn có thể kiểm soát cách thức và thời điểm Spring tạo ra các đối tượng. Dưới đây là các loại scopes chính trong Spring:
- **Singleton (Mặc định)**
 - Scope này là mặc định trong Spring. Với scope này, một bean duy nhất được tạo ra cho cả ứng dụng, bất kể số lượng yêu cầu của client. Bean được khởi tạo một lần duy nhất và tái sử dụng trong suốt vòng đời của ứng dụng.
 - **Lợi ích:**
 - Tiết kiệm bộ nhớ và tài nguyên vì chỉ có một instance duy nhất của bean.
 - **Khi nào sử dụng:**
 - Dùng khi muốn các bean là duy nhất trong toàn bộ ứng dụng.

3.1) Các loại Bean Scopes trong Spring

- ❖ **PrototypeBean** có scope Prototype sẽ được tạo mới mỗi lần yêu cầu. Mỗi khi client yêu cầu bean này, Spring sẽ tạo ra một instance mới, vì vậy không có sự chia sẻ trạng thái giữa các yêu cầu.
 - **Lợi ích:** Tạo ra một instance mới mỗi lần yêu cầu, hữu ích khi cần trạng thái độc lập cho mỗi yêu cầu. Khi nào sử dụng:
 - **Dùng khi** bạn cần mỗi yêu cầu bean có một trạng thái riêng biệt.
- ❖ **RequestBean** với scope Request sẽ được tạo ra một lần cho mỗi HTTP request. Điều này có nghĩa là một bean được tạo ra khi có một yêu cầu HTTP đến và sẽ tồn tại trong suốt quá trình xử lý yêu cầu đó.
 - **Lợi ích:** Tạo bean mới cho mỗi request, giúp đảm bảo rằng mỗi yêu cầu có một instance riêng biệt.
 - **Khi nào sử dụng:** Khi bạn cần thông tin hoặc trạng thái của một yêu cầu cụ thể (ví dụ, thông tin người dùng trong suốt quá trình xử lý một request).

3.1) Các loại Bean Scopes trong Spring

- ❖ **SessionBean** có scope Session sẽ tồn tại trong suốt vòng đời của một HTTP session. Bean này được tạo ra khi một session bắt đầu và sẽ sống trong suốt phiên làm việc của người dùng cho đến khi session kết thúc.
 - **Lợi ích:** Giúp lưu trữ và quản lý dữ liệu người dùng trong suốt thời gian session, ví dụ như lưu thông tin người dùng hoặc giỏ hàng.
 - **Khi nào sử dụng:** Khi bạn cần duy trì thông tin về người dùng hoặc trạng thái xuyên suốt các yêu cầu của một session.
- ❖ **Global Session** (Chỉ dùng trong Portlet) Scope này chỉ có tác dụng trong ứng dụng Portlet. Bean với scope Global Session sẽ tồn tại trong suốt vòng đời của session toàn cầu (tức là session trên một số trang portlet).
 - **Khi nào sử dụng:** Dùng khi làm việc với ứng dụng Portlet và cần chia sẻ session giữa các portlet khác nhau.

IV) @Autowired

- Trong Spring Framework, @Autowired là một annotation được sử dụng để tự động tiêm phụ thuộc (dependency injection) vào các thành phần của ứng dụng. Nó giúp Spring tự động xác định và tiêm các bean cần thiết vào các lớp mà không cần phải khai báo thủ công thông qua cấu hình XML hoặc các phương thức khác.
- Cách hoạt động:
 - Khi bạn gắn @Autowired vào một trường (field), phương thức setter hoặc constructor của một class, Spring sẽ tìm kiếm một bean tương thích trong container và tiêm vào đối tượng đó.
 - @Autowired có thể được sử dụng với field injection, constructor injection hoặc setter injection.

```
@Controller
public class ProductController {

    @Autowired
    private ProductService productService;

    @GetMapping("/products")
    public String getAllProducts(Model model) {
        List<Product> products = productService.findAll();
        model.addAttribute("products", products);
        return "product-list";
    }
}
```

Mối quan hệ giữa @Autowired, DI và IOC

- IOC là nguyên lý chính giúp Spring container quản lý vòng đời của các bean.
- DI là một kỹ thuật trong IOC, giúp tiêm phụ thuộc vào các đối tượng thay vì để đối tượng tự khởi tạo phụ thuộc.
- @Autowired là một công cụ trong Spring giúp thực hiện DI một cách tự động và dễ dàng. Spring container sử dụng @Autowired để tìm kiếm các bean và tiêm chúng vào các trường, constructor hoặc setter của đối tượng.

V) Các Loại Annotation trong Spring

- Trong Spring Framework, các annotation như `@Component`, `@Controller`, `@RestController`, `@Service`, và `@Repository` đều được sử dụng để đánh dấu các lớp Java làm thành phần (beans) trong Spring container.
- Các annotation này giúp Spring xác định loại và mục đích sử dụng của các lớp trong ứng dụng. Dưới đây là mô tả chi tiết về từng annotation và sự khác biệt của chúng:
- **@Component**
 - Mô tả: `@Component` là một annotation cơ bản được sử dụng để đánh dấu một lớp là một Spring Bean.
 - Lớp này sẽ được quản lý bởi Spring container, và Spring sẽ tự động phát hiện (component scanning) và tạo instance của lớp này khi ứng dụng khởi động.
 - Khi sử dụng: Dùng khi bạn muốn tạo một bean đơn giản mà không cần phải định nghĩa loại cụ thể.

```
2
3  import org.springframework.stereotype.Component;
4
5  @Component no usages new *
6  public class Config {
7      }
8  |
```

@Controller

- **Mô tả:** @Controller là một dạng chuyên dụng của @Component được sử dụng trong Spring MVC để đánh dấu một lớp là controller.
- Controller xử lý các HTTP request từ client, làm việc với các service và trả về các view hoặc dữ liệu.
- Khi sử dụng: Dùng trong ứng dụng Spring MVC, khi bạn cần một lớp để xử lý các yêu cầu HTTP từ phía client và trả về các model hoặc view.
- @Controller giúp Spring biết rằng lớp này sẽ xử lý các yêu cầu HTTP và trả về view hoặc dữ liệu từ các phương thức được đánh dấu.

```
5 @Controller ④ kiennnd25 *
6 public class ProductController {
7
8     @Autowired
9     private ProductService productService;
10
11     @GetMapping(④"/products") kiennnd25
12     public String getAllProducts(Model model) {
13         List<Product> products = productService.findAll();
14         model.addAttribute(attributeName: "products", products);
15         return "product-list";
16     }
17 }
```


@Service

- @Service là một annotation chuyên dụng để đánh dấu một lớp là một service trong ứng dụng.
- Dù @Service về cơ bản là một dạng của @Component, nó được sử dụng để làm rõ mục đích của lớp (service) và cải thiện sự hiểu biết về cấu trúc ứng dụng.
- Khi sử dụng: Dùng cho các lớp chứa logic nghiệp vụ (business logic) trong ứng dụng.

```
10
11 @Service  @kiennd25
12 public class ProductServiceImpl implements ProductService {
13
14
15     @Autowired
16     private ProductRepository productRepository;
17
18     @Override  1 usage @kiennd25
19     public List<Product> findAll() {
20         return productRepository.getAllProducts();
21     }
22 }
23
```

@Repository

- @Repository là một annotation chuyên dụng cho các lớp tương tác với cơ sở dữ liệu. Nó là một dạng của @Component, nhưng mục đích của nó là đánh dấu một lớp chịu trách nhiệm truy cập dữ liệu (DAO - Data Access Object).
- Ngoài ra, nó còn cung cấp một số tính năng đặc biệt như tự động chuyển đổi các lỗi cơ sở dữ liệu thành các ngoại lệ Spring DataAccessException. Khi sử dụng:
- Dùng khi bạn muốn đánh dấu các lớp DAO, nơi bạn thực hiện các thao tác cơ sở dữ liệu, ví dụ như sử dụng Hibernate hoặc JDBC.

```
15 @Repository 1 kienn25
16 public class ProductRepositoryImpl implements ProductRepository {
17
18     @Autowired
19     private JdbcTemplate jdbcTemplate;
20
21     // Lấy tất cả sản phẩm
22     public List<Product> getAllProducts() { 1 usage 1 kienn25
23         String sql = "SELECT * FROM products";
24         // Sử dụng RowMapper trực tiếp trong phương thức
25         return jdbcTemplate.query(sql, new RowMapper<Product>() { 1 kienn25
26             @Override no usages 1 kienn25
27             public Product mapRow(ResultSet rs, int rowNum) throws SQLException {
28                 Product product = new Product();
```

VI) JDBC Template

- JDBC Template là một lớp trong Spring Framework, thuộc Spring JDBC module, giúp đơn giản hóa việc làm việc với cơ sở dữ liệu quan hệ trong các ứng dụng Java.
- Nó là một phần của mô-đun Spring để cung cấp các phương thức tiện ích để thực hiện các truy vấn SQL mà không cần phải lo lắng về các chi tiết phức tạp của JDBC như quản lý kết nối, xử lý ngoại lệ, hoặc quản lý tài nguyên.
- JDBC Template giúp giảm thiểu mã nguồn và làm cho các thao tác cơ sở dữ liệu trở nên dễ dàng hơn bằng cách tự động xử lý các chi tiết như:
 - Mở và đóng kết nối cơ sở dữ liệu.
 - Tạo và đóng Statement hoặc PreparedStatement.
 - Quản lý các ngoại lệ SQL.
 - Xử lý các đối tượng kết quả.

6.1) Cấu hình JDBC Template trong Spring

- Để sử dụng JDBC Template trong Spring, bạn cần cấu hình DataSource (nguồn dữ liệu), sau đó tạo một instance của JdbcTemplate để thực hiện các thao tác với cơ sở dữ liệu với các cách
- 1. Cấu hình DataSource trong Spring bằng file xml trong file dispatcher-servlet.xml

```
<!-- Đọc thông tin từ file properties -->
<context:property-placeholder location="classpath:database.properties"/>
<!-- Cấu hình DataSource sử dụng Apache DBCP -->
<bean id="dataSource" class="org.apache.commons.dbcp2.BasicDataSource">
    <property name="driverClassName" value="${jdbc.driverClassName}"/>
    <property name="url" value="${jdbc.url}"/>
    <property name="username" value="${jdbc.username}"/>
    <property name="password" value="${jdbc.password}"/>
</bean>
<!-- Cấu hình JdbcTemplate -->
<bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
    <property name="dataSource" ref="dataSource"/>
</bean>
<!-- Cấu hình PlatformTransactionManager -->
<bean id="transactionManager" class="org.springframework.jdbc.datasource.DataSourceTransactionManager">
    <property name="dataSource" ref="dataSource"/>
</bean>
```

```
1 # database.properties
2 jdbc.driverClassName=com.mysql.cj.jdbc.Driver
3 jdbc.url=jdbc:mysql://localhost:3306/shop_book
4 jdbc.username=root
5 jdbc.password=root
6
```


Cấu hình JDBC Template trong Spring

- Cách 2 sử dụng config bean class application context

```
34 @Configuration new *
35 @PropertySource("classpath:database.properties") // Đọc thông tin từ file
36 public class DataSourceConfig {
37     @Value("${jdbc.driverClassName}")
38     private String driverClassName;
39     @Value("jdbc:mysql://localhost:3306/shop_book")
40     private String url;
41     @Value("root")
42     private String username;
43     @Value("root")
44     private String password;
45     @Bean new *
46     public DataSource dataSource() {
47         BasicDataSource dataSource = new BasicDataSource();
48         dataSource.setDriverClassName(driverClassName);
49         dataSource.setUrl(url);
50         dataSource.setUsername(username);
51         dataSource.setPassword(password);
52         return dataSource;
53     }
54 }
```

```
34 @Bean new *
35 public JdbcTemplate jdbcTemplate() {
36     return new JdbcTemplate(dataSource());
37 }
38
39 @Bean new *
40 public PlatformTransactionManager transactionManager() {
41     return new DataSourceTransactionManager(dataSource());
42 }
43 }
```

6.2) Sử dụng JdbcTemplate

- Khi đã cấu hình JdbcTemplate và DataSource, có thể sử dụng JdbcTemplate để thực hiện các truy vấn SQL như SELECT, INSERT, UPDATE, hoặc DELETE.

```
@Repository
public class ProductRepositoryImpl implements ProductRepository {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    // Lấy tất cả sản phẩm
    public List<Product> getAllProducts() {
        String sql = "SELECT * FROM products";
        // Sử dụng RowMapper trực tiếp trong phương thức
        return jdbcTemplate.query(sql, new RowMapper<Product>() {
            @Override
            public Product mapRow(ResultSet rs, int rowNum) throws SQLException {
                Product product = new Product();

                product.setId(rs.getInt("id"));
                product.setBookTitle(rs.getString("book_title"));
                product.setAuthor(rs.getString("author"));
                product.setPageCount(rs.getInt("page_count"));
                product.setPublisher(rs.getString("publisher"));
                product.setPublicationYear(rs.getInt("publication_year"));
                product.setGenre(rs.getString("genre"));
                product.setPrice(rs.getDouble("price"));
                product.setDiscount(rs.getDouble("discount"));
                product.setStockQuantity(rs.getInt("stock_quantity"));
            }
        });
    }
}
```


6.2) Sử dụng JdbcTemplate

- Thực hiện một truy vấn INSERT (Update)

```
// Thêm sản phẩm mới
public int addProduct(Product product) { no usages  • kiennnd25 *
    String sql = "INSERT INTO products (book_title, author, page_count, publisher, publication_year, " +
        "genre, price, discount, stock_quantity, description) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
    return jdbcTemplate.update(sql, product.getBookTitle(), product.getAuthor(), product.getPageCount(),
        product.getPublisher(), product.getPublicationYear(), product.getGenre(), product.getPrice(),
        product.getDiscount(), product.getStockQuantity(), product.getDescription());
}

// Cập nhật thông tin sản phẩm
public int updateProduct(Product product) { no usages  • kiennnd25 *
    String sql = "UPDATE products SET book_title = ?, author = ?, page_count = ?, publisher = ?, " +
        "publication_year = ?, genre = ?, price = ?, discount = ?, stock_quantity = ?, description = ? WHERE id = ?";
    return jdbcTemplate.update(sql, product.getBookTitle(), product.getAuthor(), product.getPageCount(),
        product.getPublisher(), product.getPublicationYear(), product.getGenre(), product.getPrice(),
        product.getDiscount(), product.getStockQuantity(), product.getDescription(), product.getId());
}
```


6.3) RowMapper trong JdbcTemplate

- RowMapper là một interface trong Spring JDBC được sử dụng để ánh xạ một dòng dữ liệu (row) từ kết quả của một truy vấn SQL (thường là ResultSet) thành một đối tượng Java.
- Khi sử dụng JdbcTemplate để thực hiện truy vấn, RowMapper giúp chuyển đổi mỗi dòng trong ResultSet thành một đối tượng Java tương ứng. Cách sử dụng RowMapper là rất quan trọng khi làm việc với các phương thức như query(), queryForObject(), vì nó giúp Spring hiểu cách lấy dữ liệu từ cơ sở dữ liệu và chuyển đổi nó thành các đối tượng mà bạn có thể sử dụng trong ứng dụng.
- ResultSet: Là kết quả của câu truy vấn SQL. Bạn sẽ sử dụng ResultSet để truy xuất các giá trị từ cơ sở dữ liệu.
- rowNum: Là số thứ tự của dòng hiện tại trong ResultSet (có thể không sử dụng trong nhiều trường hợp).
- T: Là kiểu dữ liệu mà bạn muốn ánh xạ kết quả SQL vào (ví dụ: User, Product, v.v.).

6.3) RowMapper trong JdbcTemplate

```
public List<Product> getAllProducts() { 1 usage  1 kiennnd25
    String sql = "SELECT * FROM products";
    // Sử dụng RowMapper trực tiếp trong phương thức
    return jdbcTemplate.query(sql, new RowMapper<Product>() { 1 kiennnd25
        @Override no usages  1 kiennnd25
        public Product mapRow(ResultSet rs, int rowNum) throws SQLException {
            Product product = new Product();

            product.setId(rs.getInt( columnLabel: "id"));
            product.setBookTitle(rs.getString( columnLabel: "book_title"));
            product.setAuthor(rs.getString( columnLabel: "author"));
            product.setPageCount(rs.getInt( columnLabel: "page_count"));
            product.setPublisher(rs.getString( columnLabel: "publisher"));
            product.setPublicationYear(rs.getInt( columnLabel: "publication_year"));
            product.setGenre(rs.getString( columnLabel: "genre"));
            product.setPrice(rs.getDouble( columnLabel: "price"));
            product.setDiscount(rs.getDouble( columnLabel: "discount"));
            product.setStockQuantity(rs.getInt( columnLabel: "stock_quantity"));
            product.setDescription(rs.getString( columnLabel: "description"));

            return product;
        }
    });
}
```



VIỆN CÔNG NGHỆ THÔNG TIN T3H

Đào tạo chuyên sâu - Trải nghiệm thực tế

www.t3h.edu.vn

© Copyright 2023 GV Nguyễn Đức Kiên



VIỆN CÔNG NGHỆ THÔNG TIN T3H

Đào tạo chuyên sâu - Trải nghiệm thực tế

www.t3h.edu.vn

© Copyright 2023 GV Nguyễn Đức Kiên



VIỆN CÔNG NGHỆ THÔNG TIN T3H

Đào tạo chuyên sâu - Trải nghiệm thực tế

www.t3h.edu.vn

© Copyright 2023 GV Nguyễn Đức Kiên



VIỆN CÔNG NGHỆ THÔNG TIN T3H

Đào tạo chuyên sâu - Trải nghiệm thực tế

www.t3h.edu.vn

© Copyright 2023 GV Nguyễn Đức Kiên

